

Design Pattern - 2

↳ strategy

↳ ocp

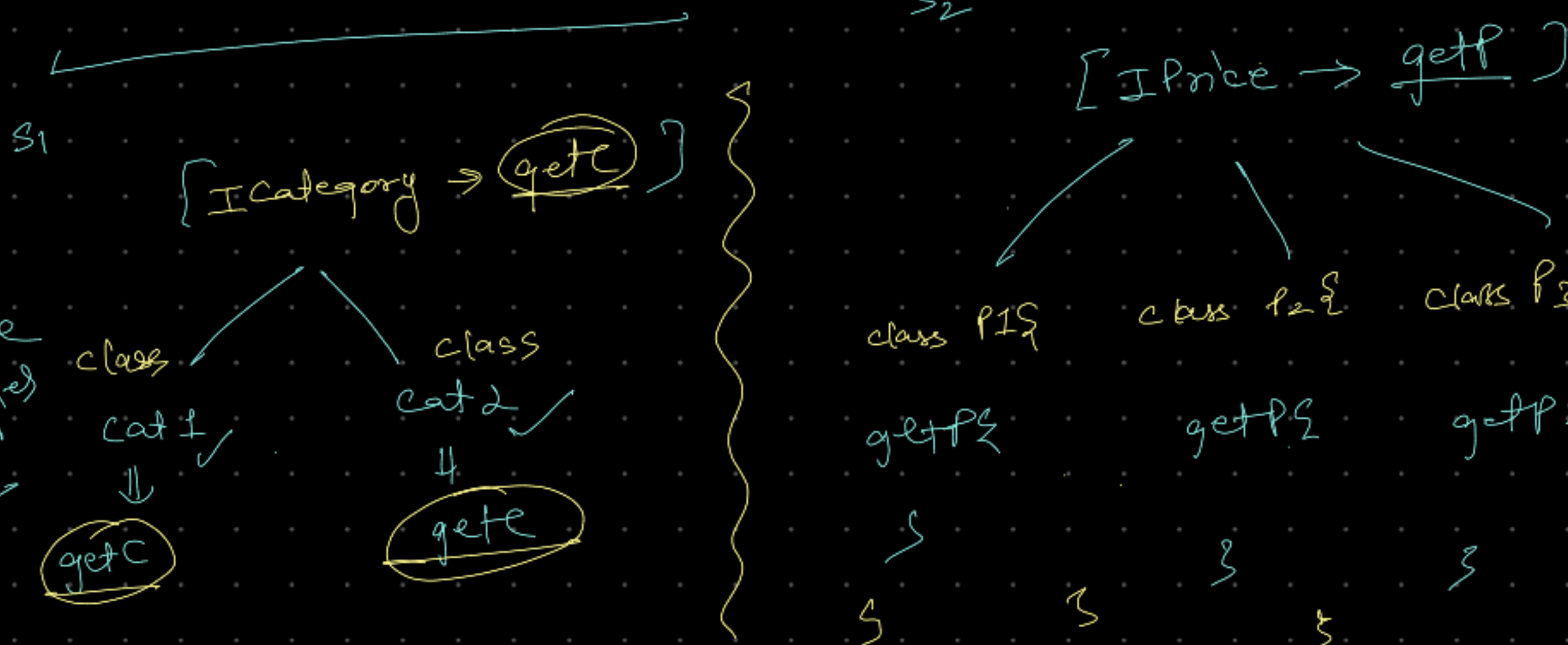
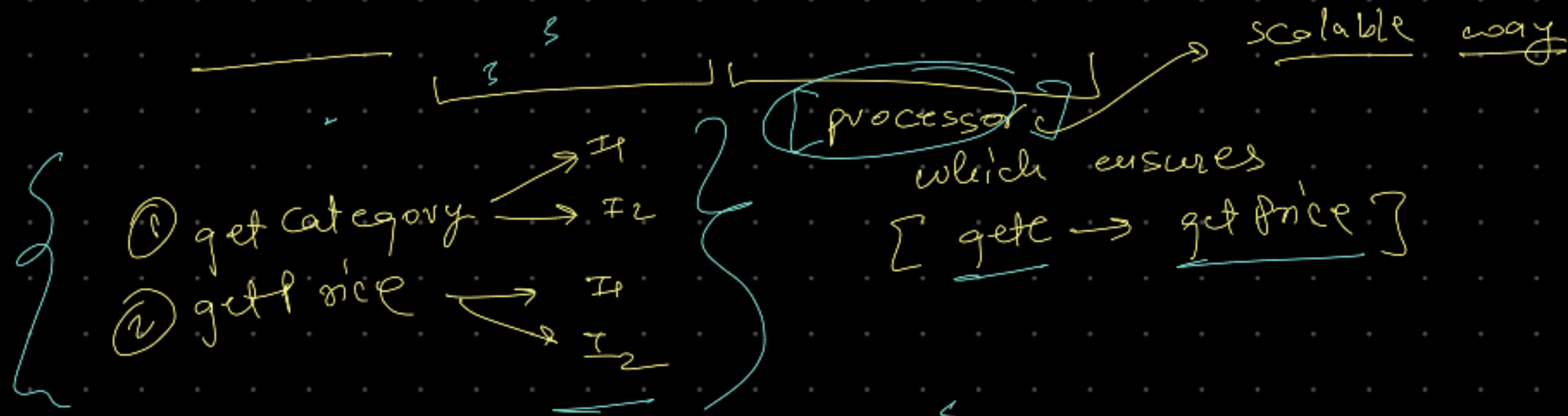
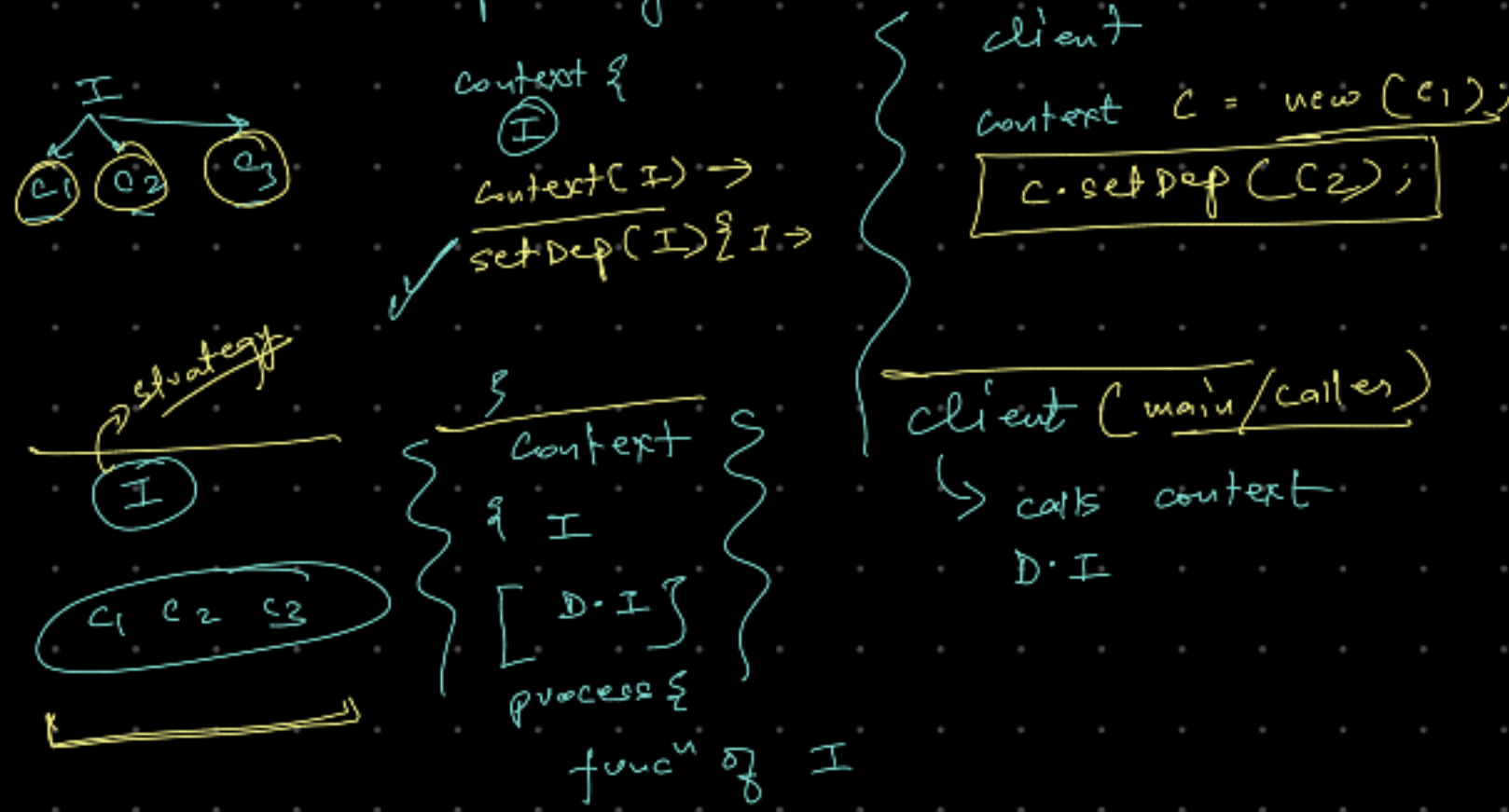
↳ switch b/w algo at runtime.

splitwise

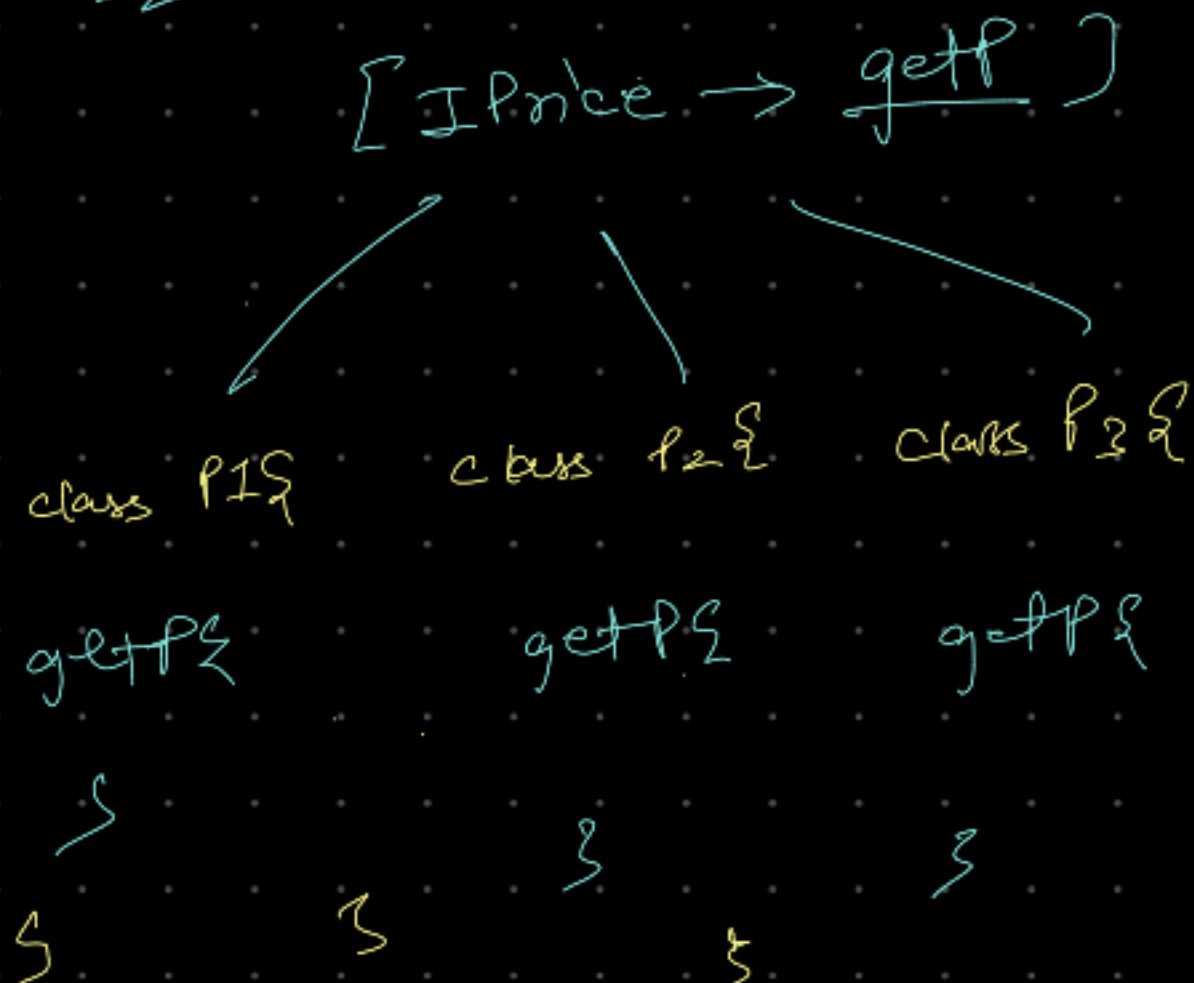
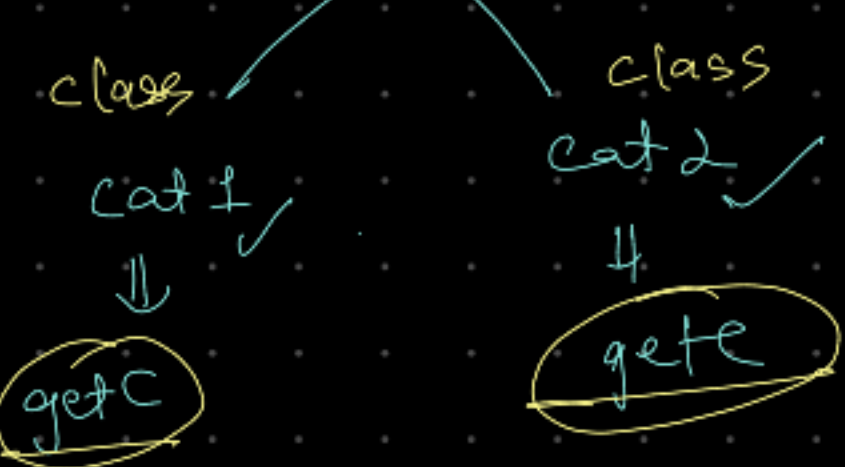
shapes #/✓

↳ constructor injection → Depn
↳ setter for the dependency

strategy → should be able to switch the dependency at runtime ✓



concrete strategies for S1



concrete strategies for S2

Context class
↓
client would be using

class Processor {

I.Cate → ic

I.Price → ip

[Processor(I.Cat, I.P)]

setC(I.C) ~

setP(I.P) ~

process {

ic.getc()

ip.getp()

}

ensuring the order

[Runtime polymorphism]

client → {

Processor p1

p1.setC(C1)

p1.setP(P1, P2)

p1.process();

}

client

client → context

injecting the depⁿ
[concrete strategies]